# CS 499 Milestone Two Narrative

*Software Design and Engineering Enhancement*

## Artifact Description

The artifact is the Travlr Getaways application created in CS 465 Full Stack Development. It is a MEAN-stack travel application with an Express and Node.js server, a MongoDB database, REST API routes, a public travel website, and an Angular administrator application. The original project was created during the CS 465 course and was later expanded during the final modules of that course. The application allows users to view trips while authenticated administrators can add, update, and delete trip records.

## Reason for Including the Artifact

I chose Travlr Getaways because it shows the type of software I want to work with. The project includes a public travel website and an administrator application. I worked with Express, Angular, MongoDB, REST APIs, and user authentication. The administrator can manage trip information through the application.

The artifact was improved by separating important responsibilities that had previously been mixed together. Authentication is now handled by reusable middleware in app_api/middleware/authenticate.js. Trip input validation is handled by validateTrip.js. Database operations are organized in tripService.js instead of being placed directly in the controller. This makes the code easier to read, test, and maintain. The controllers now focus more on responding to requests and handling application results.

The enhancement also improves security. Protected POST, PUT, and DELETE routes require a bearer token. Registration trims and normalizes email addresses before saving them. Error responses no longer return raw database error details to the client. This reduces the amount of internal information exposed by the API. The project also includes tests for missing trip fields, complete trip validation, missing bearer tokens, and valid JWT authentication.

## Course Outcome Progress

This enhancement really does support Outcome 4 as I've applied software engineering practices to an existing application. The key thing I did was to separate out the authentication, validation and database layers which made the code a lot easier to maintain and test. The backend tests were also very focused and provided a good way of reliably testing the key bits of behaviour.

The enhancement supports Outcome 5 via bearer token protection, input validation, normalised registration emails and safer error responses. It also supports Outcome 2 as I've explained the design decisions and the test results in both my code review and in this narrative. Outcome 1 is supported to a degree as the code review and the feedback from the instructor did help me to consider how another developer would go about understanding and maintaining the code. A lot of the implementation was done individually so I've used my own experience and other examples from the course to demonstrate the collaborative element. Outcome 3 is not really the focus of this artefact as LumberTrack is a much more direct demonstration of my algorithm and data structure skills.

My instructor felt the enhancement met with expectations and suggested we go further and do more field validation and more automated testing to cover a wider range of application behaviours and edge cases. The current test suite does cover the key validation and auth scenarios. More integration testing and better field level validation are the areas we'd still like to improve upon.

## Reflection on the Enhancement Process

The main lesson from this enhancement was that working code is not automatically well-structured code. The original application performed important tasks, but authentication and validation were located in places that made future changes harder. Moving those responsibilities into middleware made the request flow easier to understand. Moving database operations into a service also made the controller less dependent on the database implementation.

One challenge was making improvements without changing the expected behavior of the Angular application and existing API routes. I had to preserve route names, request methods, response status codes, and the fields used by the trip model. Another challenge was testing code that normally depends on MongoDB and the complete application environment. I addressed this by adding focused backend tests for reusable middleware. The tests provide a repeatable way to verify important security and validation behavior.

Planning the changes first made the work easier. I knew which files needed to be changed and what I needed to test. I also learned that security needs to be included while the application is being built.